

Concise Study Guide: Algorithms (Chapter 3)

Based on slides for Chapter 3

Section 3.1: Algorithms

What is an Algorithm?

An algorithm is a finite sequence of precise, unambiguous instructions for performing a computation or solving a problem.

- It takes **input** from a specified set.
- It produces **output** from a specified set (the solution).
- **Correctness**: It must produce the correct output for all valid inputs.
- **Finiteness**: It must terminate after a finite number of steps.
- **Effectiveness**: Each step must be basic enough to be doable in a finite time.
- **Generality**: It should work for all problems of the specified form.

Example

Problem: Find the maximum value in a finite sequence of integers.

Algorithm (in English):

1. Set a "temporary maximum" to the first integer in the sequence.
2. Compare the next integer to the "temporary maximum".
3. If this integer is larger, set it as the new "temporary maximum".
4. Repeat from step 2 until there are no more integers in the sequence.
5. The "temporary maximum" is your answer.

Specifying Algorithms

Algorithms can be written in English, but to be more precise, we often use **pseudocode**. Pseudocode is a structured, programming-like language that is not tied to any specific syntax (like Java or C++). It helps us analyze an algorithm's efficiency.

Pseudocode

```
procedure max( $a_1, a_2, \dots, a_n$ : integers) max :=  $a_1$  for  $i$  := 2 to  $n$  if max <  $a_i$  then max :=  $a_i$   
return max
```

Question

Does the 'max' algorithm above have all the properties of an algorithm (Input, Output, Correctness, Finiteness, Effectiveness, Generality)? **Answer:** Yes, it does.

Searching Algorithms

Problem: Find an element x in a list $a_1, a_2, \dots, a_n$.

1. Linear Search

Examines elements one by one, from the beginning.

Pseudocode

```
procedure linear_search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : distinct integers)  $i := 1$  while ( $i \leq n$  and  $x \neq a_i$ )  $i := i + 1$  if  $i \leq n$  then location :=  $i$  else location := 0 return location
```

2. Binary Search

Requirement: The list must be sorted (e.g., in increasing order). This algorithm repeatedly divides the search interval in half.

1. Compare x with the middle element.
2. If x is larger, search the upper half of the list.
3. If x is smaller, search the lower half of the list.
4. Repeat until the list has size 1 or x is found.

Pseudocode

```
procedure binary_search( $x$ : integer,  $a_1, a_2, \dots, a_n$ : increasing integers)  $i := 1$  {left endpoint}  
 $j := n$  {right endpoint} while  $i < j$   $m := \lfloor (i + j)/2 \rfloor$  if  $x > a_m$  then  $i := m + 1$  else  $j := m$  if  
 $x = a_i$  then location :=  $i$  else location := 0 return location
```

Example

Search for 19 in the list: [1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22]

1. Midpoint is at pos 8 (value 10). $19 > 10$. Search list from pos 9 to 16.

2. Midpoint is at pos 12 (value 16). $19 > 16$. Search list from pos 13 to 16.

3. Midpoint is at pos 14 (value 19). $19 \neq 19$. Search list from pos 13 to 14.

4. Midpoint is at pos 13 (value 18). $19 > 18$. Search list from pos 14 to 14.

5. Loop ends ($i = 14, j = 14$). Check if $x = a_i$. $19 = 19$. Return location 14.

Sorting Algorithms

Problem: Put a list of elements into increasing order.

1. Bubble Sort

Makes multiple passes. In each pass, it compares adjacent elements and "bubbles" the largest element to its correct position at the end.

Pseudocode

```
procedure bubblesort( $a_1, \dots, a_n$ : real numbers) for  $i := 1$  to  $n - 1$  for  $j := 1$  to  $n - i$  if  $a_j > a_{j+1}$  then interchange  $a_j$  and  $a_{j+1}$ 
```

Example

Sort the list: 3 2 4 1 5

- **First Pass:**

- (3 2 4 1 5) $\rightarrow$ (2 3 4 1 5) $\rightarrow$ (2 3 4 1 5) $\rightarrow$ (2 3 1 4 5)
- Result: (2 3 1 4 5)

- **Second Pass:**

- (2 3 1 4 5) $\rightarrow$ (2 3 1 4 5) $\rightarrow$ (2 1 3 4 5)
- Result: (2 1 3 4 5)

- **Third Pass:**

- (2 1 3 4 5) $\rightarrow$ (1 2 3 4 5)
- Result: (1 2 3 4 5)

- **Fourth Pass:**

- (1 2 3 4 5)
- Result: (1 2 3 4 5)

Final Sorted List: 1 2 3 4 5

2. Insertion Sort

Builds the final sorted list one item at a time. It takes each element and "inserts" it into its correct position in the already-sorted part of the list.

Pseudocode

```
procedure insertion_sort( $a_1, \dots, a_n$ : real numbers) for  $j := 2$  to  $n$   $i := 1$  while  $a_j > a_i$   $i := i + 1$   $m := a_j$  for  $k := 0$  to  $j - i - 1$   $a_{j-k} := a_{j-k-1}$   $a_i := m$ 
```

Example

Sort the list: 3 2 4 1 5

- ([3] 2 4 1 5) $\rightarrow$ ([2 3] 4 1 5) {Insert 2}
- ([2 3] 4 1 5) $\rightarrow$ ([2 3 4] 1 5) {Insert 4}
- ([2 3 4] 1 5) $\rightarrow$ ([1 2 3 4] 5) {Insert 1}
- ([1 2 3 4] 5) $\rightarrow$ ([1 2 3 4 5]) {Insert 5}

Final Sorted List: 1 2 3 4 5

Greedy Algorithms

An algorithmic paradigm (a general approach) that makes the "best" or "most optimal" choice at each step, hoping to find a global optimum. **Warning:** A greedy choice at one step doesn't always lead to the best overall solution.

Example

Problem: Make change for n cents using the fewest total coins (U.S. coins: 25c, 10c, 5c, 1c).

Greedy Idea: At each step, choose the coin with the largest possible value that doesn't go over the remaining amount.

Example ($n = 67$ cents):

1. Choose 25c. Remaining: 42c.
2. Choose 25c. Remaining: 17c.
3. Choose 10c. Remaining: 7c.
4. Choose 5c. Remaining: 2c.
5. Choose 1c. Remaining: 1c.
6. Choose 1c. Remaining: 0c.

Total coins: 6 (2 quarters, 1 dime, 1 nickel, 2 pennies). This is optimal for U.S. coins.

Pseudocode

```
procedure change( $c_1, \dots, c_r$ : coin values;  $n$ : positive integer) for  $i := 1$  to  $r$   $d_i := 0$   
{count for coin  $c_i$ } while  $n \geq c_i$   $d_i := d_i + 1$   $n := n - c_i$  return ( $d_1, \dots, d_r$ )
```

Question (Optimality): Does this greedy algorithm *always* work?

- It works for U.S. coins (25, 10, 5, 1).
- **Counterexample:** Consider coins {25c, 10c, 1c} and $n = 31$ cents.
 - **Greedy Solution:** 1x25c, 6x1c = **7 coins**. (Corrected count from previous)
 - **Optimal Solution:** 3x10c, 1x1c = **4 coins**.
- The greedy approach failed because the "best" local choice (25c) prevented the optimal global solution.

Example

Problem: Greedy Scheduling. Schedule as many talks as possible in one lecture hall. No two talks can overlap.

Greedy Choices:

1. Pick the talk that starts earliest? (Doesn't work. A long talk might start early and block many shorter talks).
2. Pick the shortest talk? (Doesn't work. A short talk might overlap two other talks).
3. **Pick the talk that ends earliest.** (This one works!)

Pseudocode

```
procedure schedule( $s_1, \dots, s_n$ : start times,  $e_1, \dots, e_n$ : end times) Sort talks by finish  
time so  $e_1 \leq e_2 \leq \dots \leq e_n$   $S := \emptyset$  {set of scheduled talks} for  $j := 1$  to  $n$  if talk  $j$  is  
compatible with all talks in  $S$  then  $S := S \cup \{\text{talk } j\}$  return  $S$ 
```

The Halting Problem

Problem: Can we create an algorithm (let's call it $H(P, I)$) that takes any program P and its input I and determines if P will eventually halt (stop) or loop forever? **Answer:** No. This problem is **unsolvable**.

Proof (by contradiction):

1. **Assume** $H(P, I)$ exists. It outputs "halts" or "loops forever".
2. Now, construct a new, clever procedure $K(P)$ that uses H as a helper:
 - $K(P)$ first runs $H(P, P)$. (It feeds the program P to itself as its own input).
 - **If** $H(P, P)$ outputs "halts", **then** $K(P)$ enters an infinite loop.
 - **If** $H(P, P)$ outputs "loops forever", **then** $K(P)$ halts.
3. K does the *opposite* of whatever H predicts P will do.
4. **The Contradiction:** What happens when we run $K(K)$? (We feed K to itself).
 - **Case 1:** Assume $K(K)$ halts.
 - By the definition of K , this can only happen if $H(K, K)$ outputted "loops forever".
 - But H is supposed to be a perfect predictor. If H says $K(K)$ loops forever, it must loop forever.
 - This is a contradiction. $K(K)$ cannot halt.
 - **Case 2:** Assume $K(K)$ loops forever.
 - By the definition of K , this can only happen if $H(K, K)$ outputted "halts".
 - But H is a perfect predictor. If H says $K(K)$ halts, it must halt.
 - This is also a contradiction. $K(K)$ cannot loop forever.
5. Since both possibilities lead to a contradiction, our initial assumption was wrong. The procedure H cannot exist.

Section 3.2: The Growth of Functions

We use "Asymptotic Notation" to describe how fast an algorithm's runtime (or memory use) grows as the input size (n) gets very large. We care about the *rate of growth*, not the exact number of steps.

Big-O Notation (Upper Bound)

Definition: We say $f(x)$ is $O(g(x))$ if there are positive constants C (a multiplier) and k (a starting point) such that:

$$|f(x)| \leq C \cdot |g(x)| \quad \text{whenever } x > k$$

In English: " $f(x)$ grows **no faster than** $g(x)$ (times some constant C) for all large values of x (after $x = k$)."
 $g(x)$ is an **upper bound** on the growth of $f(x)$.

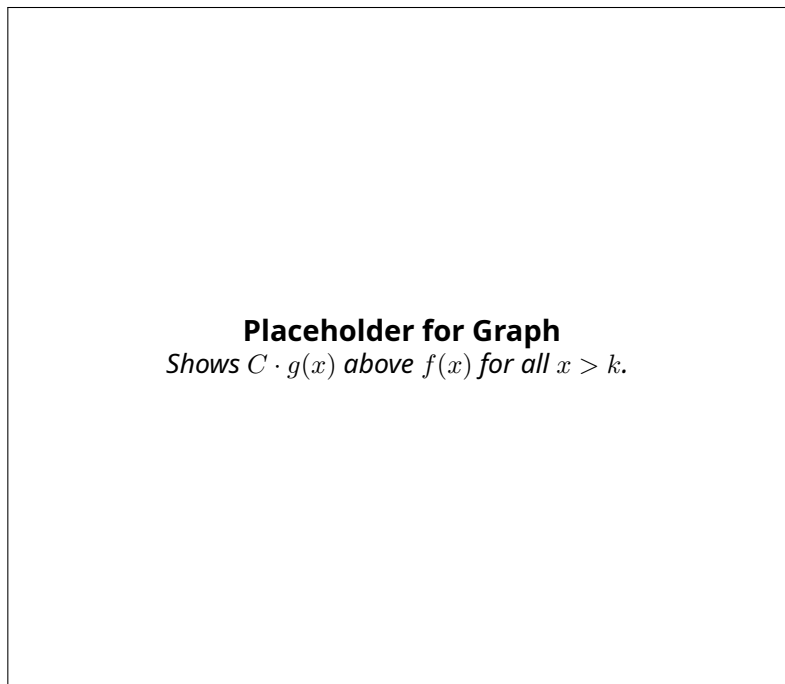


Figure 1: Big-O Notation Illustration

Example

Show that $f(x) = x^2 + 2x + 1$ **is** $O(x^2)$. We need to find C and k such that $x^2 + 2x + 1 \leq C \cdot x^2$ for $x > k$.

- Let's try $k = 1$. For $x > 1$:
- $x < x^2$ (e.g., $2 < 4$)
- $1 < x^2$ (e.g., $1 < 4$)
- So, $x^2 + 2x + 1 < x^2 + 2x^2 + 1x^2 = 4x^2$
- We found $C = 4$ and $k = 1$.
- Therefore, $x^2 + 2x + 1$ is $O(x^2)$.

Example

Show that $7x^2$ is $O(x^3)$. We need $7x^2 \leq C \cdot x^3$.

- If we divide by x^2 (assuming $x > 0$), we get $7 \leq C \cdot x$.
- Let's pick $C = 1$. We need $7 \leq x$.
- This is true for all $x > 7$. So, $C = 1$ and $k = 7$ are witnesses.

Example

Show that n^2 is *not* $O(n)$.

- **Proof (by contradiction):** Assume n^2 is $O(n)$.
- Then there must be constants C and k such that $n^2 \leq C \cdot n$ for all $n > k$.
- Divide by n (assuming $n > 0$): $n \leq C$.
- This says that n (which can be any number larger than k) must always be less than or equal to some fixed constant C .
- This is impossible. We can always pick a number n that is bigger than C (e.g., $n = \max(k, C) + 1$).
- This is a contradiction. Therefore, n^2 is not $O(n)$.

Big-O for Polynomials: A polynomial $f(x) = a_n x^n + \dots + a_1 x + a_0$ is $O(x^n)$. The term with the highest power dominates the growth.

Big-Omega (Ω) Notation (Lower Bound)

Definition: We say $f(x)$ is $\Omega(g(x))$ if there are positive constants C and k such that:

$$|f(x)| \geq C \cdot |g(x)| \quad \text{whenever } x > k$$

In English: " $f(x)$ grows at least as fast as $g(x)$." $g(x)$ is a **lower bound**.

Example

Show that $f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(x^3)$. We need to find C and k such that $8x^3 + 5x^2 + 7 \geq C \cdot x^3$ for $x > k$.

- For $x > 1$, $5x^2 > 0$ and $7 > 0$.
- So, $8x^3 + 5x^2 + 7 > 8x^3$.
- We can choose $C = 8$ and $k = 1$.
- Therefore, $f(x)$ is $\Omega(x^3)$.

Big-Theta (Θ) Notation (Tight Bound)

Definition: We say $f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ AND $f(x)$ is $\Omega(g(x))$. **In English:** " $f(x)$ grows at the same rate as $g(x)$." $g(x)$ is a **tight bound**.

This means $f(x)$ is "sandwiched" between two versions of $g(x)$: $C_1 \cdot |g(x)| \leq |f(x)| \leq C_2 \cdot |g(x)|$ for all $x > k$.

Example

Show that $f(n) = 1 + 2 + \dots + n$ **is** $\Theta(n^2)$. We know $f(n) = \frac{n(n+1)}{2} = \frac{1}{2}n^2 + \frac{1}{2}n$.

1. **Show** $O(n^2)$:

- $\frac{1}{2}n^2 + \frac{1}{2}n \leq \frac{1}{2}n^2 + \frac{1}{2}n^2 = 1 \cdot n^2$ (for $n > 1$)
- So, $f(n)$ is $O(n^2)$ with $C_2 = 1, k = 1$.

2. **Show** $\Omega(n^2)$:

- $\frac{1}{2}n^2 + \frac{1}{2}n \geq \frac{1}{2}n^2$ (for $n > 0$)
- So, $f(n)$ is $\Omega(n^2)$ with $C_1 = 1/2, k = 1$.

Since $f(n)$ is $O(n^2)$ and $\Omega(n^2)$, it is $\Theta(n^2)$.

Big-Theta for Polynomials: A polynomial $f(x) = a_n x^n + \dots + a_0$ (where $a_n \neq 0$) is $\Theta(x^n)$.

Question

Ordering Functions by Growth Rate Put the following functions in order, from slowest-growing to fastest-growing.

- $f_1(n) = (1.5)^n$
- $f_2(n) = 8n^3 + 17n^2 + 111$
- $f_3(n) = (\log n)^2$
- $f_4(n) = 2^n$
- $f_5(n) = \log(\log n)$
- $f_6(n) = n^2(\log n)^3$
- $f_7(n) = 2^n(n^2 + 1)$
- $f_8(n) = n^3 + n(\log n)^2$
- $f_9(n) = 10000$
- $f_{10}(n) = n!$

Solution (Slowest to Fastest):

1. $f_9(n) = 10000$ (Constant, $\Theta(1)$)
2. $f_5(n) = \log(\log n)$ (Log-log)
3. $f_3(n) = (\log n)^2$ (Log-squared)
4. $f_6(n) = n^2(\log n)^3$
5. $f_2(n) = 8n^3 + 17n^2 + 111$ (Polynomial $\Theta(n^3)$)
6. $f_8(n) = n^3 + n(\log n)^2$ (Tied with f_2 , also $\Theta(n^3)$)
7. $f_1(n) = (1.5)^n$ (Exponential)
8. $f_4(n) = 2^n$ (Exponential, base $2 > 1.5$)
9. $f_7(n) = 2^n(n^2 + 1)$ ($\Theta(n^2 2^n)$)
10. $f_{10}(n) = n!$ (Factorial, grows faster than any exponential)

Section 3.3: Complexity of Algorithms

Time Complexity: How many operations an algorithm uses (as a function of input size n). **Space Complexity:** How much memory an algorithm uses. We focus on **worst-case time complexity:** the maximum number of operations for any input of size n .

Complexity Analysis of Algorithms

Example

Algorithm: 'max($a_1, \dots, a_n$)'

- **Operation:** Comparisons.
- The loop runs from $i = 2$ to n , which is $n - 1$ times.
- Inside the loop, one comparison ('max < a_i ') is made.
- The loop itself makes one comparison per step to check $i \leq n$ (plus one final check to exit).
- Total comparisons is $\approx (n - 1) + n = 2n - 1$.
- **Complexity:** The number of operations is $\Theta(n)$. This is **linear complexity**.

Example

Algorithm: Linear Search

- **Operation:** Comparisons.
- **Worst-Case:** The item x is not in the list (or is at the very end).
- The loop runs n times in the worst case.
- Inside the loop, two comparisons are made ($i \leq n$ and $x \neq a_i$).
- One final check ($i \leq n$) when the loop condition fails.
- One more comparison ($i \leq n$) after the loop.
- Total comparisons in worst case: $2n + 2$.
- **Worst-Case Complexity:** $\Theta(n)$.

Average-Case: Assuming x is in the list and equally likely to be anywhere, the average number of comparisons is $\frac{1}{n} \sum_{i=1}^n (2i + 1) = \frac{2}{n} \frac{n(n+1)}{2} + \frac{n}{n} = (n + 1) + 1 = n + 2$. Still $\Theta(n)$.

Example

Algorithm: Binary Search

- **Operation:** Comparisons.
- **Worst-Case:** The list size is n . After the comparisons in one step, the search interval size is roughly halved ($n/2$). This continues until the size is 1.
- The number of times you can divide n by 2 until you reach 1 is $\approx \log_2 n$.
- Let $n = 2^k$, it takes $k = \log_2 n$ iterations.
- Inside the 'while' loop, 2 comparisons are made ($i < j$ and $x > a_m$). One comparison ($i < j$) to exit. One comparison ($x = a_i$) after loop.
- Total comparisons $\approx 2 \log_2 n + 2$.
- **Worst-Case Complexity:** $\Theta(\log n)$. This is **logarithmic complexity**, which is much, much faster than $\Theta(n)$.

Example

Algorithm: Bubble Sort

- **Operation:** Comparisons ($a_j > a_{j+1}$).
- The outer loop ('i') runs from 1 to $n - 1$.
- The inner loop ('j') runs from 1 to $n - i$.
- Total comparisons: $\sum_{i=1}^{n-1} (n - i) = (n - 1) + (n - 2) + \dots + 2 + 1 = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$.
- **Worst-Case Complexity:** $\Theta(n^2)$. This is **polynomial complexity**.

Example

Algorithm: Insertion Sort

- **Operation:** Comparisons (primarily ' $a_j > a_i$ ').
- **Worst-Case:** The list is in reverse order.
- The outer loop ('j') runs from 2 to n .
- In the worst case, the inner 'while' loop compares a_j with all $j - 1$ elements before it ($i = 1, \dots, j - 1$), plus one comparison to exit. So j comparisons in the j -th iteration.
- Total comparisons in worst case: $\sum_{j=2}^n j = (2 + 3 + \dots + n) = \frac{n(n+1)}{2} - 1$.
- **Worst-Case Complexity:** $\Theta(n^2)$.

Example

Algorithm: Matrix Multiplication (for two $n \times n$ matrices)

- **Operation:** Scalar multiplications and additions.
- To compute one entry c_{ij} in the product matrix C : $c_{ij} = \sum_{q=1}^n a_{iq}b_{qj}$.
- This requires n multiplications and $n - 1$ additions.
- There are n^2 total entries in the matrix C .
- Total multiplications = $n^2 \cdot n = n^3$.
- Total additions = $n^2 \cdot (n - 1) = n^3 - n^2$.
- **Complexity:** The number of arithmetic operations is $\Theta(n^3)$.

Question

Matrix-Chain Multiplication How should we parenthesize the product $A_1A_2A_3$ to use the fewest multiplications?

- A_1 is 30×20
- A_2 is 20×40
- A_3 is 40×10

(Note: Multiplying an $m \times k$ matrix by a $k \times n$ matrix takes $m \cdot k \cdot n$ multiplications.)

Solution:

1. Case 1: $(A_1A_2)A_3$

- A_1A_2 : (30×20) by $(20 \times 40) \rightarrow 30 \cdot 20 \cdot 40 = 24,000$ mults. Result is 30×40 .
- $(A_1A_2)A_3$: (30×40) by $(40 \times 10) \rightarrow 30 \cdot 40 \cdot 10 = 12,000$ mults.
- **Total:** $24,000 + 12,000 = 36,000$ **multiplications**.

2. Case 2: $A_1(A_2A_3)$

- A_2A_3 : (20×40) by $(40 \times 10) \rightarrow 20 \cdot 40 \cdot 10 = 8,000$ mults. Result is 20×10 .
- $A_1(A_2A_3)$: (30×20) by $(20 \times 10) \rightarrow 30 \cdot 20 \cdot 10 = 6,000$ mults.
- **Total:** $8,000 + 6,000 = 14,000$ **multiplications**.

Answer: The second method, $A_1(A_2A_3)$, is more than twice as efficient.

Common Complexity Classes

Table 1: Commonly Used Terminology for the Complexity of Algorithms.

Complexity	Terminology
$\Theta(1)$	Constant complexity
$\Theta(\log n)$	Logarithmic complexity
$\Theta(n)$	Linear complexity
$\Theta(n \log n)$	Linearithmic complexity
$\Theta(n^b)$	Polynomial complexity
$\Theta(b^n)$, where $b > 1$	Exponential complexity
$\Theta(n!)$	Factorial complexity

Table 2: The Computer Time Used by Algorithms (Assuming 1 op = 10^{-11} sec).

Problem Size	Bit Operations Used						
	n	log n	n	n log n	n ²	2 ⁿ	n!
10	3 × 10 ⁻¹¹ s	10 ⁻¹⁰ s	3 × 10 ⁻¹⁰ s	10 ⁻⁹ s	10 ⁻⁸ s	3 × 10 ⁻⁷ s	
10 ²	7 × 10 ⁻¹¹ s	10 ⁻⁹ s	7 × 10 ⁻⁹ s	10 ⁻⁷ s	4 × 10 ¹¹ yr	*	*
10 ³	1.0 × 10 ⁻¹⁰ s	10 ⁻⁸ s	1 × 10 ⁻⁷ s	10 ⁻⁵ s	*	*	*
10 ⁴	1.3 × 10 ⁻¹⁰ s	10 ⁻⁷ s	1 × 10 ⁻⁶ s	10 ⁻³ s	*	*	*
10 ⁵	1.7 × 10 ⁻¹⁰ s	10 ⁻⁶ s	2 × 10 ⁻⁵ s	0.1 s	*	*	*
10 ⁶	2 × 10 ⁻¹⁰ s	10 ⁻⁵ s	2 × 10 ⁻⁴ s	0.17 min	*	*	*

(* = more than 10^{100} years)

Tractable vs. Intractable Problems

- **Tractable Problem:** A problem that can be solved by an algorithm with **polynomial time** complexity (e.g., $\Theta(n^b)$ for some constant b). These are in **Class P**.
- **Intractable Problem:** A problem for which no polynomial time algorithm exists. Exponential ($\Theta(b^n)$) and factorial ($\Theta(n!)$) complexities often indicate intractability for large n .
- **Unsolvable Problem:** No algorithm exists to solve it (e.g., the Halting Problem).
- **Class NP:** (Nondeterministic Polynomial time) A class of problems where a proposed solution can be **verified** in polynomial time.
- **Class NP-Complete:** The "hardest" problems in NP. If a polynomial-time algorithm is found for *any* NP-Complete problem, then *all* problems in NP can be solved in polynomial time (meaning $P=NP$).

The P vs. NP Problem: This is a major unsolved question in computer science: **Is $P = NP$?** In simpler terms: "If the answer to a problem can be checked quickly (in polynomial time), can the answer also be found quickly (in polynomial time)?" Most computer scientists believe $P \neq NP$, but a formal proof is missing. Finding such a proof (or disproof) carries a \$1,000,000 prize from the Clay Mathematics Institute.